VERERBUNG: „IST EIN“

Unterklasse übernimmt Attribute und Methoden der Oberklasse und ergänzt Neues. Sparkonto **ist ein** Konto → Vererbung. Konto **hat einen** Kunden → Attribut.

```
class Konto:
    def __init__(self, inhaber, stand=0):
        self.inhaber = inhaber
        self.stand = stand

    def einzahlen(self, betrag):
        self.stand += betrag

    def abheben(self, betrag):
        if betrag > self.stand:
            return False
        self.stand -= betrag
        return True

class Sparkonto(Konto):   # erbt
    def __init__(self, inh, stand, zins):
        super().__init__(inh, stand)
        self.zins = zins   # neu

s = Sparkonto("A", 1000, 2)
s.einzahlen(500)          # geerbt
print(s.stand, s.zins)    # 1500 2
```

Merksatz: Eigenes `__init__` → `super().__init__(…)` ist Pflicht, Python ruft es nicht selbst auf.

`isinstance(s, Konto)` → True (zählt Unterklassen mit), `type(s) is Konto` → False (nur genaue Klasse).

ÜBERSCHREIBEN

Gleicher Name, gleiche Parameter, in der Unterklasse.

```
from typing import override

class Girokonto(Konto):
    def __init__(self, inh, stand, dispo):
        super().__init__(inh, stand)
        self.dispo = dispo

    @override
    def abheben(self, betrag):
        if betrag > self.stand + self.dispo:
            return False
        self.stand -= betrag
        return True

g = Girokonto("B", 100, 500)
print(g.abheben(400))  # True, stand -300
print(g.abheben(300))  # False
```

- Suche: Klasse des Objekts → Oberklasse → … → `object`. Erste Fundstelle gewinnt (`__mro__`).
- `super().methode()` benutzt die Version der Oberklasse mit.
- `@override`: Python ignoriert es, ty findet damit Tippfehler.

Merksatz: **Überladen** (gleicher Name, andere Parameter, dieselbe Klasse) gibt es in Python nicht, **Überschreiben** schon.

POLYMORPHIE

Derselbe Aufruf, verschiedenes Verhalten je nach Klasse des Objekts.

```
konten = [Konto("A", 100),
          Girokonto("B", 100, 500)]
for k in konten:
    print(k.abheben(400))
# False (Konto) / True (Girokonto)
```

Duck Typing: Python prüft beim Ausführen nur, ob die Methode existiert, nicht die Klasse.

OBJECT UND ABSTRAKTE KLASSEN

Jede Klasse erbt von `object` (Standard-`__str__`, `__repr__`, `__eq__`).

```
from abc import ABC, abstractmethod

class Zahlungsart(ABC):
    @abstractmethod
    def bezahlen(self) -> str: ...

Zahlungsart()  # TypeError
```

- Keine Objekte einer abstrakten Klasse, Fehler schon **beim Erzeugen**.
- Jede Unterklasse muss alle `@abstractmethod` überschreiben, sonst bleibt sie abstrakt.

FÄHIGKEITEN STATT INTERFACES

Python hat kein `interface`, erlaubt aber mehrere Oberklassen: erst Kategorie, dann Fähigkeiten.

```
class Druckbar(ABC):
    @abstractmethod
    def drucken(self) -> None: ...

class Dokument: ...

class Lieferschein(Dokument, Druckbar):
    @override
    def drucken(self) -> None:
        print("druckt")

d = Lieferschein()
if isinstance(d, Druckbar):
    d.drucken()  # druckt
```

- **Kategorie** („ist ein“): konkrete und abstrakte Methoden, Attribute, Konstruktor, eine je Klasse.
- **Fähigkeit** („kann“): nur abstrakte Methoden, beliebig viele je Klasse.

TYPISCHE FEHLER

- `super().__init__` vergessen: `AttributeError` erst bei der ersten Methode.
- `super.__init__(…)` ohne Klammern hinter `super`.
- Unterklasse überschreibt Attribut der Oberklasse (`self.stand = laufzeit`).
- Import von `override`, `ABC`, `abstractmethod` vergessen (`NameError`) oder `(ABC)` vergessen.
- Fähigkeit ohne `isinstance` genutzt: `AttributeError`.